

Увод в програмирането - основни конструкции

Ивайло Андреев + gemini 3 Flash, chat GPT 5.4

16 май 2026

Съдържание

1	Принципи на структурното програмиране	2
2	Управление на изчислителния процес	2
2.1	Условни оператори	2
2.2	Оператори за цикъл	3
3	Променливи	3
3.1	Локални и глобални променливи	4
3.2	Инициализация и оператор за присвояване	4
4	Функции и процедури	4
4.1	Предаване на параметри	4
4.2	Типове и проверка за съответствие на тип	5

1 Принципи на структурното програмиране

Процедурната парадигма служи като основа за разбирането на съвременните подходи и е неизменна част от изучаването на алгоритми и структури от данни. Програмирането може да бъде разделено на две основни парадигми:

- **Императивно програмиране:** Описва *как* се изпълнява дадена задача, чрез поредица от инструкции, които променят състоянието на програмата (например чрез променливи и команди за контрол на потока). То включва процедурното и обектно-ориентираното програмиране.
- **Декларативно програмиране:** Описва *какво* трябва да се постигне, без изрично да се указват стъпките за изпълнение (например функционално програмиране и логическо програмиране).

Фундаментална теорема на структурното програмиране (Теорема на Бьом-Якопини): Всеки алгоритъм може да бъде изразен чрез използването на само три основни управляващи конструкции:

1. **Последователност (Sequence):** Инструкциите се изпълняват една след друга в реда, в който са написани.
2. **Разклонение (Selection/Condition):** Избор между два или повече пътя на изпълнение в зависимост от логическо условие.
3. **Цикъл (Iteration/Loop):** Многократно изпълнение на поредица от инструкции, докато дадено условие е изпълнено.

Основната цел на структурното програмиране е да направи програмите по-разбираеми, по-лесни за проследяване и модификация, като се избягва използването на неструктурирани преходи като `goto`.

2 Управление на изчислителния процес

Управлението на изчислителния процес се осъществява чрез инструкции, които определят реда на изпълнение на действията. Програмата се състои от изрази (които пресмятат и връщат стойност) и оператори (които извършват действия). Блок от код се огражда с къдрави скоби `{}`.

2.1 Условни оператори

Условните оператори позволяват разклоняване на логиката. Използват се `if`, `if-else` и `switch` за многозначно разклоняване.

```
1 // Example of conditional statements and basic syntax
2 int x = 10;
3 if (x > 5) {
4     // Executed if the condition is true
```

```

5     x--;
6 } else {
7     // Executed if the condition is false
8     x++;
9 }
10
11 // Inline ternary conditional choice
12 int y = (x == 9) ? 1 : 0;
13
14 // Switch selector statement
15 switch (y) {
16     case 0:
17         // Handle case 0 block
18         break; // Stop fall-through
19     case 1:
20         // Handle case 1 block
21         break;
22     default:
23         // Optional catch-all block
24         break;
25 }

```

2.2 Оператори за цикъл

Циклите осигуряват многократно изпълнение на код. Те се делят на цикли с предусловие (while), цикли с постусловие (do-while) и цикли с брояч (for).

```

1 // 1. Pre-condition loop (evaluates condition before running body)
2 int i = 0;
3 while (i < 5) {
4     // Loop code statements
5     i++;
6 }
7
8 // 2. Post-condition loop (guaranteed to execute at least once)
9 int j = 0;
10 do {
11     // Executes body, then evaluates condition
12     j++;
13 } while (j < 5);
14
15 // 3. Counter-controlled loop iteration
16 for (int k = 0; k < 5; k++) {
17     // Initialization, condition check, and step execution
18 }

```

3 Променливи

Променливата е наименувана област от паметта, която съхранява стойност от определен тип. Всяка променлива има адрес, съответстващ на позицията ѝ в паметта.

3.1 Локални и глобални променливи

Основните разлики между променливите се свеждат до техния обхват (видимост) и време на живот:

- **Локални променливи:** Дефинират се вътре във функция или блок `{}`. Те се създават при достигане на дефиницията им и се унищожават при излизане от блока (разполагат се в **Стека**). Видими са само в този блок.
- **Глобални променливи:** Дефинират се извън всички функции, в началото на файла. Те съществуват през цялото време на изпълнение на програмата (разположени в **Data/BSS сегмента**) и са достъпни от всяка функция в рамките на файла.

3.2 Инициализация и оператор за присвояване

- **Оператор за присвояване (=):** Променя текущата стойност на вече съществуваща променлива, като оценява израза отдясно и го записва в паметта на променливата отляво.
- **Инициализация:** Задаване на начална стойност на променливата в момента на нейното деклариране. Важно е да се отбележи, че неинициализираните локални променливи съдържат произволни данни от паметта (т.нар. "бок-лук"), което води до грешки и недефинирано поведение, докато глобалните се инициализират автоматично с 0.

4 Функции и процедури

Функцията е самостоятелен блок от код, който решава подзадача и може да бъде извикан многократно. В C++ процедурите се представят като функции с тип на връщане `void`.

- **Формални параметри:** Променливите, изброени в заглавието на функцията.
- **Фактически параметри (Аргументи):** Реалните стойности или променливи, които се подават в скобите при извикването.

4.1 Предаване на параметри

По официален конспект се разглеждат следните основни механизми за предаване:

- **Предаване по стойност (Pass by Value):** Създава се локално копие на аргумента в стековата рамка на функцията. Промените вътре във функцията не влияят на оригиналната променлива.

- **Предаване по име (Pass by Name):** Исторически модел (използван например в ALGOL 60), при който аргументът се подставя на местата на формалния параметър в тялото на функцията при извикване. Този механизъм не е част от стандартния C++, но се разглежда теоретично в конспекта.

Забележка: За да се променят оригиналните стойности на променливите, в практиката на C++ се използва предаване чрез псевдоним (референция) или указател, но теоретичният акцент в академичния конспект пада върху горните два модела.

```
1 // Demonstrating parameter passing models
2 void modifyValue(int num) {
3     // Pass by value: changes only the local stack copy
4     num = 100;
5 }
6
7 void modifyByReference(int& num) {
8     // Pass by reference: changes the original argument
9     num = 100;
10 }
11
12 int main() {
13     int x = 5;
14
15     modifyValue(x);           // x remains 5
16     modifyByReference(x);     // x becomes 100
17
18     return 0;
19 }
```

4.2 Типове и проверка за съответствие на тип

C++ е статично типизиран език. Компиляторът извършва строга проверка за съответствие на типовете (**Type Checking**) по време на компилация. Когато подадем аргумент на функция, компилаторът проверява дали типът му съвпада с дефинирания формален параметър. При несъвпадение се прави опит за автоматично (неявно) преобразуване на типа, а ако такова е невъзможно, компилацията се прекъсва с грешка за несъвместимост.